

1. (i) The following is a scatter plot consisting of 186 data points along with two principal directions.

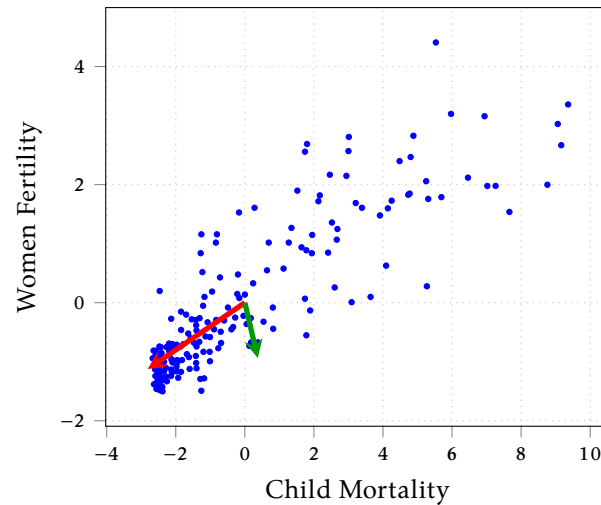


Figure 1. Women fertility as a function of neonatal child mortality.

Based on **Figure 1**, the direction captured by the vector in green (along the coordinates $[0.3747, -0.9271]$) carries the maximal amount of variance across the 186 observations, and thus, it encodes more information to that of the vector/direction in red. It is therefore reasonable to conclude that the direction of the vector in green is more important.

- (ii) The following is an approximation of the mortality/fertility dataset with the best Rank-1 approximation.

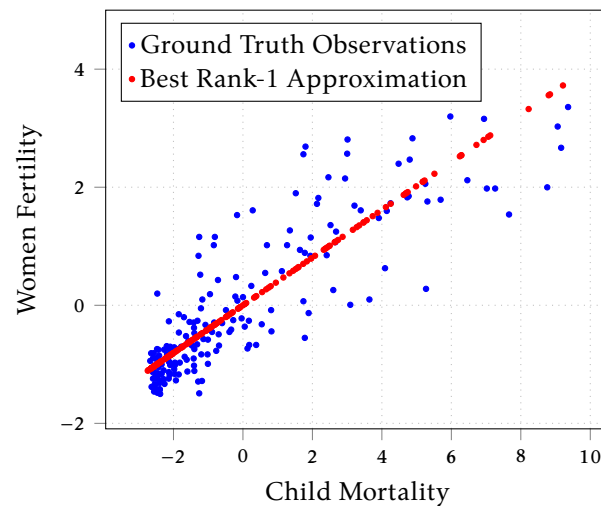


Figure 2. Women fertility as a function of neonatal child mortality.
Best Rank-1 approximation.

2. (A) (i) Below is a plot with ε on the x -axis and the mean absolute differences on the y -axis corresponding to the gaussian random projection subroutine.

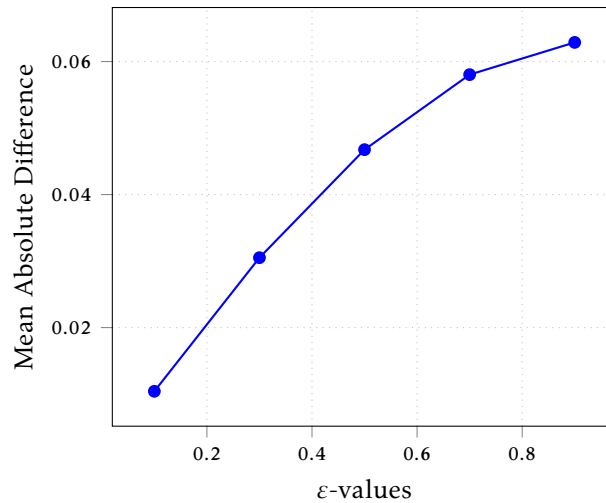


Figure 3. Mean absolute difference of gaussian random projection.

(ii) Below is a histogram corresponding to the absolute differences.

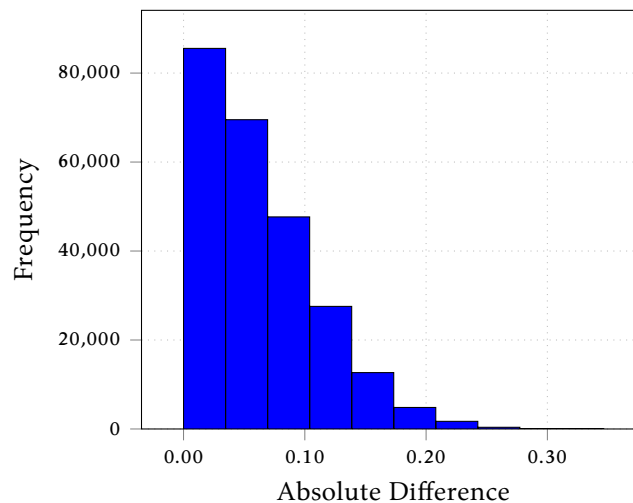


Figure 4. Histogram of absolute differences of gaussian random projection.

(iii) Yes, from mean absolute difference plot and absolute differences histogram, we see that pairwise distances are nearly preserved.

(iv) Below is the implementation:

```

1  import numpy as np
2  import sklearn.datasets
3
4  from sklearn.random_projection import GaussianRandomProjection
5  from sklearn.random_projection import johnson_lindenstrauss_min_dim
6  from sklearn.metrics.pairwise import euclidean_distances
7
8  np.random.seed(42)
9
10 def preprocess(size: int = 500) → np.ndarray:
```

```

11 data = sklearn.datasets.fetch_rcv1().data
12 idxs = np.random.choice(data.shape[0], size)
13 data = data[idxs, :]
14 return data
15
16 def main() → int:
17     data = preprocess()
18     eps_vals = []
19     abs_diff = []
20
21     for eps in np.arange(0.1, 0.999, 0.2):
22         min_dimension = johnson_lindenstrauss_min_dim(data.shape[0], eps=eps)
23         if data.shape[1] < min_dimension:
24             continue
25         gaussian_projection = GaussianRandomProjection(eps=eps)
26         data_transform = gaussian_projection.fit_transform(data)
27         original_distances = euclidean_distances(data)
28         tranform_distances = euclidean_distances(data_transform)
29         absolute_difference = np.abs(original_distances - tranform_distances)
30         abs_diff.append(absolute_difference.mean())
31         eps_vals.append(eps)
32
33     np.savetxt('data/gauss-proj-abs-diff.csv', absolute_difference.flatten(), delimiter=',')
34     np.savetxt('data/gauss-proj-eps-vs-mean-abs-diff.csv', np.column_stack((eps_vals, abs_diff)),
35               delimiter=',')
36
37     return 0
38
39 if __name__ == '__main__':
40     raise SystemExit(main())

```

Listing 1. gauss-rand-proj.py

- (B) (i) Below is a plot with ε on the x -axis and the mean absolute differences on the y -axis corresponding to the sparse random projection subroutine.

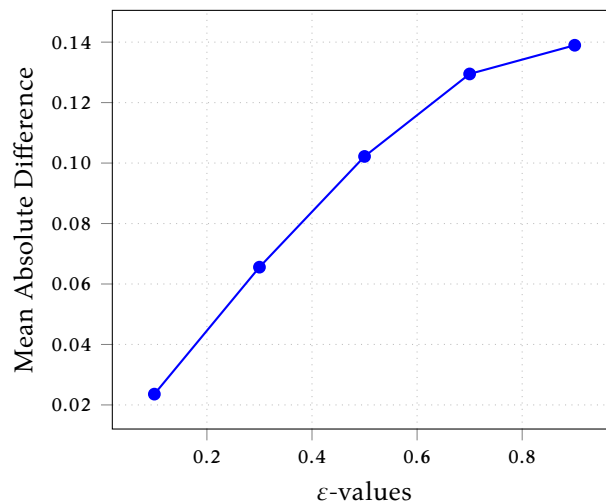


Figure 5. Mean absolute difference of sparse random projection.

- (ii) Below is a histogram corresponding to the absolute differences.

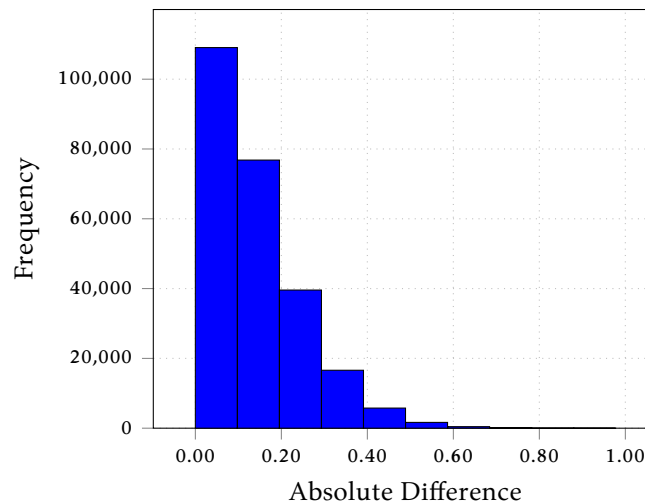


Figure 6. Histogram of absolute differences of sparse random projection.

(iii) Yes, from the mean absolute difference plot and absolute differences histogram, we see that pairwise distances are nearly preserved.

(iv) Below is the implementation:

```

1  import numpy as np
2  import sklearn.datasets
3
4  from sklearn.random_projection import SparseRandomProjection
5  from sklearn.random_projection import johnson_lindenstrauss_min_dim
6  from sklearn.metrics.pairwise import euclidean_distances
7
8  np.random.seed(42)
9
10 def preprocess(size: int = 500) → np.ndarray:
11     data = sklearn.datasets.fetch_rcv1().data
12     idxs = np.random.choice(data.shape[0], size)
13     data = data[idxs, :]
14     return data
15
16 def main() → int:
17     data = preprocess()
18     eps_vals = []
19     abs_diff = []
20
21     for eps in np.arange(0.1, 0.999, 0.2):
22         min_dimension = johnson_lindenstrauss_min_dim(data.shape[0], eps=eps)
23         if data.shape[1] < min_dimension:
24             continue
25         sparse_projection = SparseRandomProjection(eps=eps)
26         data_transform = sparse_projection.fit_transform(data)
27         original_distances = euclidean_distances(data)
28         transform_distances = euclidean_distances(data_transform)
29         absolute_difference = np.abs(original_distances - transform_distances)
30         abs_diff.append(absolute_difference.mean())
31         eps_vals.append(eps)
32
33     np.savetxt('data/sparse-proj-abs-diff.csv', absolute_difference.flatten(), delimiter=',')
34     np.savetxt('data/sparse-proj-eps-vs-mean-abs-diff.csv', np.column_stack((eps_vals, abs_diff)),
35               , delimiter=',')
36     return 0

```

```
37  
38 if __name__ == '__main__':  
39     raise SystemExit(main())
```

Listing 2. sparse-rand-proj.py